ЛАБОРАТОРНА РОБОТА № 4

Тема: Модульне програмування, низькорівневі порозрядні операції, бітові маски та командний процес розробки через Pull Request у Git

1. МЕТА РОБОТИ Опанувати принципи функціональної декомпозиції програм: проектування чистих підпрограм із чітким розділенням на прототипи (оголошення) та реалізацію (визначення). Дослідити механізм роботи автоматичних локальних змінних на стеку та використання специфікатора static для збереження внутрішнього стану функцій між послідовними викликами. Опанувати низькорівневе маніпулювання даними на рівні окремих розрядів за допомогою порозрядних операцій & , | , ^ , ~ , << , >> . Засвоїти чотири канонічні інженерні операції над апаратними регістрами мікропроцесора: встановлення, скидання, інвертування та тестування стану бітів за допомогою динамічно сформованих масок. Опанувати командний процес інтеграції коду через створення, рецензування та злиття Pull Request (PR) на платформі GitHub. Підготувати підсумкову інженерну документацію та долучити обов'язкову пояснювальну записку у форматі PDF до звітності.

ХІД РОБОТИ:

Етап 1. Функціональна декомпозиція та збереження стану

Лістинг

11.Функція обчислення полінома . Статична змінна рахує кількість проходів графіка через нуль ( ).

```c
#include <stdio.h>

double calculatePolynomial(double a, double b, double c, double x);

double calculatePolynomial(double a, double b, double c, double x)
{
    static int zeroCount = 0;

    double y = a * x * x + b * x + c;

    if (y == 0.0)
    {
        zeroCount++;
    }

    printf("Number of zero results: %d\n", zeroCount);

    return y;
}

int main(void)
{
    int n;

    double a, b, c, x, y;

    printf("Polynomial: Y = AX^2 + BX + C\n");
    printf("Enter number of calculations: ");

    if (scanf("%d", &n) != 1 || n <= 0 || n > 100)
    {
        printf("Invalid input!\n");
        return 1;
    }

    for (int i = 0; i < n; i++)
    {
        printf("\nCalculation %d\n", i + 1);

        printf("Enter A: ");
        scanf("%lf", &a);

        printf("Enter B: ");
        scanf("%lf", &b);

        printf("Enter C: ");
        scanf("%lf", &c);

        printf("Enter X: ");
        scanf("%lf", &x);

        y = calculatePolynomial(a, b, c, x);

        printf("Result Y = %.2f\n", y);
    }

    return 0;
}
```

```
Polynomial: Y = AX^2 + BX + C
Enter number of calculations: 1

Calculation 1
Enter A: -3
Enter B: 2
Enter C: 1
Enter X: 4
Number of zero results: 0
Result Y = -39.00
```

Етап 2. Емуляція 32-бітного апаратного регістра GPIO (task2.c)

11.Встановити біт 12; скинути біт 0; інвертувати біт 5; перевірити стан біта 22.

Лістинг

```c
#include <stdio.h>
#include <stdint.h>

uint32_t set_bit(uint32_t reg, unsigned int n);
uint32_t clear_bit(uint32_t reg, unsigned int n);
uint32_t toggle_bit(uint32_t reg, unsigned int n);
int check_bit(uint32_t reg, unsigned int n);
void print_binary(uint32_t reg);

uint32_t set_bit(uint32_t reg, unsigned int n)
{
    return reg | (1u << n);
}

uint32_t clear_bit(uint32_t reg, unsigned int n)
{
    return reg & ~(1u << n);
}

uint32_t toggle_bit(uint32_t reg, unsigned int n)
{
    return reg ^ (1u << n);
}

int check_bit(uint32_t reg, unsigned int n)
{
    return (reg & (1u << n)) != 0;
}

void print_binary(uint32_t reg)
{
    for (int i = 31; i >= 0; i--)
    {
        printf("%u", (reg >> i) & 1u);

        if (i % 4 == 0)
        {
            printf(" ");
        }
    }

    printf("\n");
}

int main(void)
{
    uint32_t reg = 0x00400021;

    printf("32-bit GPIO register emulation\n");

    printf("\nInitial register:\n");
    print_binary(reg);

    printf("\n1. Set bit 12:\n");
    printf("Before: ");
    print_binary(reg);

    reg = set_bit(reg, 12);

    printf("After:  ");
    print_binary(reg);

    printf("\n2. Clear bit 0:\n");
    printf("Before: ");
    print_binary(reg);

    reg = clear_bit(reg, 0);

    printf("After:  ");
    print_binary(reg);

    printf("\n3. Toggle bit 5:\n");
    printf("Before: ");
    print_binary(reg);

    reg = toggle_bit(reg, 5);

    printf("After:  ");
    print_binary(reg);

    printf("\n4. Check bit 22:\n");
    printf("Register: ");
    print_binary(reg);

    if (check_bit(reg, 22))
    {
        printf("Bit 22 is SET (1).\n");
    }
    else
    {
        printf("Bit 22 is CLEAR (0).\n");
    }

    return 0;
}
```

```
32-bit GPIO register emulation

Initial register:
0000 0000 0100 0000 0000 0000 0010 0001

1. Set bit 12:
Before: 0000 0000 0100 0000 0000 0000 0010 0001
After:  0000 0000 0100 0000 0001 0000 0010 0001

2. Clear bit 0:
Before: 0000 0000 0100 0000 0001 0000 0010 0001
After:  0000 0000 0100 0000 0001 0000 0010 0000

3. Toggle bit 5:
Before: 0000 0000 0100 0000 0001 0000 0010 0000
After:  0000 0000 0100 0000 0001 0000 0000 0000

4. Check bit 22:
Register: 0000 0000 0100 0000 0001 0000 0000 0000
Bit 22 is SET (1).
```

Етап 3. Пакування та розпакування телеметрії в бітове слово (task3.c)

11.[Біти 0..3] Версія протоколу (0..15); [Біти 4..11] Довжина корисного пакета (0..255); [Біти 12..15] Контрольна сума CRC (0..15).

Лістинг:

```c
#include <stdio.h>
#include <stdint.h>

uint16_t pack_data(unsigned int version,
                   unsigned int length,
                   unsigned int crc);

void unpack_data(uint16_t data,
                 unsigned int *version,
                 unsigned int *length,
                 unsigned int *crc);

uint16_t pack_data(unsigned int version,
                   unsigned int length,
                   unsigned int crc)
{
    return (uint16_t)(
        (version & 0x0F) |
        ((length & 0xFF) << 4) |
        ((crc & 0x0F) << 12)
    );
}

void unpack_data(uint16_t data,
                 unsigned int *version,
                 unsigned int *length,
                 unsigned int *crc)
{
    *version = data & 0x0F;
    *length = (data >> 4) & 0xFF;
    *crc = (data >> 12) & 0x0F;
}

int main(void)
{
    unsigned int version = 5;
    unsigned int length = 150;
    unsigned int crc = 10;

    unsigned int restoredVersion;
    unsigned int restoredLength;
    unsigned int restoredCrc;

    uint16_t packet;

    printf("Telemetry packet packing\n");

    printf("\nOriginal data:\n");
    printf("Protocol version: %u\n", version);
    printf("Packet length: %u\n", length);
    printf("CRC: %u\n", crc);

    packet = pack_data(version, length, crc);

    printf("\nPacked packet:\n");
    printf("HEX: 0x%04X\n", packet);

    unpack_data(packet,
                &restoredVersion,
                &restoredLength,
                &restoredCrc);

    printf("\nRestored data:\n");
    printf("Protocol version: %u\n", restoredVersion);
    printf("Packet length: %u\n", restoredLength);
    printf("CRC: %u\n", restoredCrc);

    if (version == restoredVersion &&
        length == restoredLength &&
        crc == restoredCrc)
    {
        printf("\nData verification: SUCCESS\n");
    }
    else
    {
        printf("\nData verification: FAILED\n");
    }

    return 0;
}
```

```
Telemetry packet packing

Original data:
Protocol version: 5
Packet length: 150
CRC: 10

Packed packet:
HEX: 0xA965

Restored data:
Protocol version: 5
Packet length: 150
CRC: 10

Data verification: SUCCESS
```